

Product-CRUD-API

4.Product CRUD API

Create Product module:

Fields:

- id
- name
- price
- qty

APIs:

- POST /product/add
- GET /product/list
- PUT /product/update/{id}
- DELETE /product/delete/{id}

Use ArrayList instead of DB.

Solution :

This project demonstrates the implementation of a Product Management Module using Java and Spring Boot. The application provides CRUD (Create, Read, Update, Delete)

operations for managing product details such as product ID, product name, price, and quantity. To keep the application simple, product data is stored in an ArrayList instead of a database. The application exposes REST APIs that allow users to add new products, view all products, update existing product details, and delete products. This project helps in understanding RESTful API development, collection handling, and basic CRUD operations in Java.

Java code:

Product Model

```
public class Product {  
    private int id;  
    private String name; private double price; private int qty; public Product() {  
    }  
    public Product(int id, String name, double price, int qty) {  
        this.id = id;  
        this.name = name;  
        this.price = price;  
        this.qty = qty;  
    }  
    public int getId() {
```

```

return id;
}
public void setId(int id) {
this.id = id;
}

public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}
public double getPrice() {
return price;
}
public void setPrice(double price) {
this.price = price;
}
public int getQty() {
return qty;
}
public void setQty(int qty) {
this.qty = qty;
}
}

```

Product Controller

```

import org.springframework.web.bind.annotation.*; import java.util.ArrayList;
import java.util.List;
@RestController
@RequestMapping("/product")

public class ProductController {
private List<Product> productList = new ArrayList<>(); @PostMapping("/add")
public String addProduct(@RequestBody Product product) {
productList.add(product);
return "Product Added Successfully"; }
@GetMapping("/list")
public List<Product> getAllProducts() {
return productList; }
@PutMapping("/update/{id}")
public String updateProduct(@PathVariable int id,
@RequestBody Product updatedProduct) { for (Product product : productList) {

```

```

if (product.getId() == id) { product.setName(updatedProduct.getName());
product.setPrice(updatedProduct.getPrice()); product.setQty(updatedProduct.getQty()); return
"Product Updated Successfully";
}}
return "Product Not Found"; }
@DeleteMapping("/delete/{id}")
public String deleteProduct(@PathVariable int id) {

for (Product product : productList) { if (product.getId() == id) {
productList.remove(product);
return "Product Deleted Successfully"; }
}
return "Product Not Found"; }
}

```

API Endpoints

Method	API
POST	/product/add
GET	/product/list
PUT	/product/update/{id}
DELETE	/product/delete/{id}

Output :

```

POST /product/add
Product Added Successfully GET /product/list
[
{
  "id": 1,
  "name": "Pen",
  "price": 20,
  "qty": 100
}]

```